Detection and segmentation

COCO

LECTURE 14 | GÉRON CH. 12

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Three lectures of vision, and every one of them answers the same question: *what is in this image?*

- Lecture 12 — a convolutional network gives one label per image
- Lecture 13 — a pretrained backbone gives a much better one, cheaply

> **WHAT NEITHER CAN EXPRESS**
>
> **Where** the object is, and **how many** there are. A photograph with three cyclists in it has one label and three answers, and nothing in the architecture can say so.

Today the output stops being a label and becomes a set of boxes — which means the first thing we need is a new notion of what a *correct answer* is. That is the derivation.

Today

1. The task, the corpus, and what a detector returns (15 min)
2. **The mathematics** — IoU, its vanishing gradient, and mAP as a mean of a mean (25 min)
3. **The method** — running a detector, choosing a threshold, non-maximum suppression (30 min)
4. Segmentation, and tracking briefly (15 min)

The derivation carries a debt from Lecture 3: average precision is defined with a *maximum* in it, and that maximum exists precisely to repair the non-monotonicity of precision proved there.

PART ONE

The brief, and why the classifier cannot answer it

15 minutes

The stakeholder

> **RETAIL SHELF AUDIT**
>
> “We photograph shelves. We need to know *how many* of each product are on each shelf, and *where* each one is, so that a picking route can be planned. A list of the categories present is no use to us.”

- Two questions, not one: **how many** and **where**
- The second question is what changes the architecture

Naming and locating are different problems

	Classification	Detection
Question	what is in this picture	what is where, and how many
Output	one vector, length K	three arrays, length N
N is	✓ **fixed before you look**	✗ **decided by the image and by two cut-offs**
Wanted from pooling	✓ **invariance**	✗ **equivariance**

The last row is Lecture 12 with a consequence. A network that has learned to *not care* where the object is cannot tell you where it is.

What a detector returns, next to what a classifier returns

Classification

one image
3 × H × W
the input, either way

→

CNN backbone
Lectures 12–13
frozen, pretrained

→

Output: one vector of length K
"this image is a cat, 0.93"
Shape is fixed before you look at the image.
One label per image; nothing says where.

Detection

Region proposals
a few hundred candidate
boxes, scored and refined

→

Suppression
near-duplicates removed
by overlap — today

→

Output: three arrays, of a length nobody chose
boxes (N × 4) · labels (N) · scores (N)
N depends on the image, on a score cut-off, and on an overlap cut-off.
Two numbers you did not choose are already in your answer.

Two numbers you did not choose are already inside the answer on the right: a score cut-off and an overlap cut-off.

A box, precisely

Four numbers. Which four is not agreed on, and that is the first bug.

Convention	Meaning	Used by
`[x1, y1, x2, y2]`	two opposite corners	torchvision, and everything today
`[x, y, w, h]`	top-left corner, then size	the COCO annotation file
`[cx, cy, w, h]`	centre, then size	YOLO-family papers

THIS WILL BITE YOU TODAY

The two you will hold in the same notebook are the first two. Read `[100, 50, 20, 30]` as corners and you get a box 20 wide when it should be 20 *starting at* 100 — a valid box, in the wrong place, with no error.

PART ONE, CONTINUED

The corpus

and exactly how big it is

COCO

- *Common Objects in Context* — the standard detection benchmark, and the set these pretrained weights were trained on
- 80 object categories, drawn by hand, in everyday scenes rather than studio shots
- Every object carries a box, a category, and a polygon outline
- The validation split, `val2017`, is 5,000 images

The textbook names COCO as the dataset the pretrained detection and segmentation models in this chapter are trained on.

Our corpus is 128 images. Not COCO

128

images, out of **5,000** in `val2017`

- The 128 numerically lowest image ids — a rule, not a selection
- About 20 MB of JPEG, plus the official annotation file
- Full COCO with the training split is roughly 20 GB and is not downloaded anywhere in this course

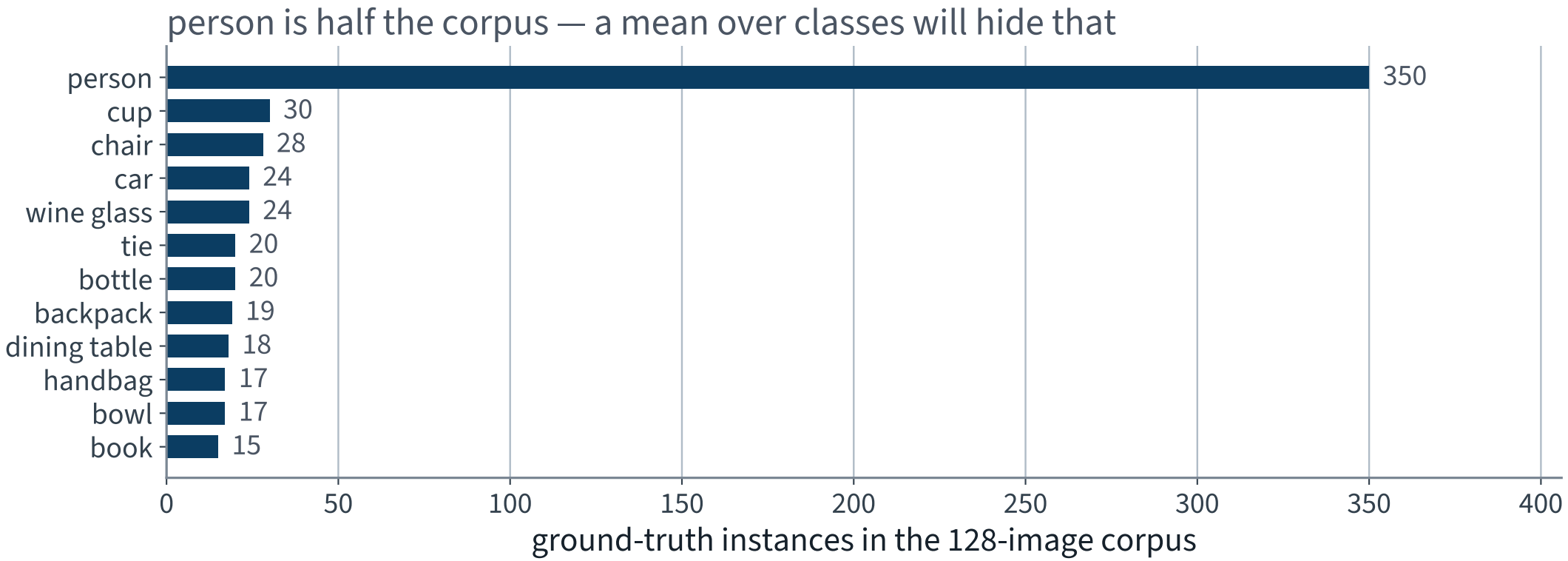
✗ Every number in these two lectures is measured on 128 images. When we say a score, we say the sample size beside it.

The corpus, counted

Quantity	Value
Images	128
Annotated objects, crowd regions excluded	898
Crowd regions, dropped	8
Distinct categories appearing	73 of 80
Objects per image — mean	7.02
Objects per image — median	5
Objects per image — range	1 to 29

Mean 7.02 against median 5: the distribution has a long right tail, and one image alone carries 29 objects.

Twelve categories carry most of it



39.0% of every annotated object in this corpus is a person. Remember that when we start averaging over categories.

Scope, stated up front

- **Examinable:** what detection is, what a detector returns, intersection over union and average precision, semantic against instance segmentation — Géron, Chapter 12
- **Not examinable:** the internal architecture of Faster R-CNN, the torchvision model-zoo API, COCO's file format, anchor design

You will use all four of the non-examinable items today. Using a tool and being examined on it are different things, and the course says which is which every time.

THE MATHEMATICS

Scoring a box, scoring a detector

25 minutes · examinable

One quantity, four requirements

Before writing anything down, fix what it has to do. Whatever we write must:

1. be 1 for coincident boxes and 0 for disjoint ones
2. punish a box for being too large
3. punish a box for being too small
4. be dimensionless, so a cup and a sofa are on one scale

> **THE CONSTRUCTION**
>
> Requirement 2 says the *predicted* area must be in the denominator. Requirement 3 says the *true* area must be too. Requirement 4 says the numerator must be an area as well. There is essentially one candidate.

Intersection over union

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

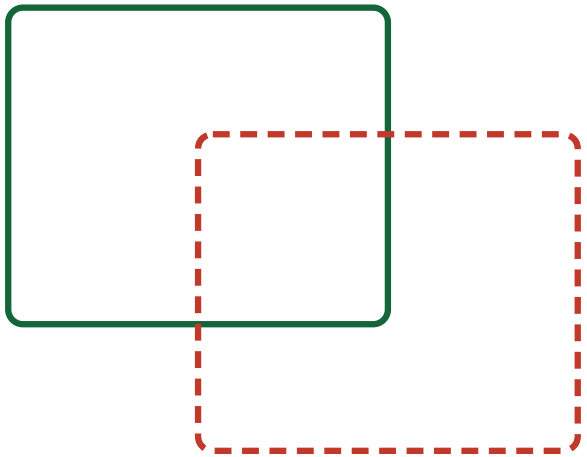
ALSO CALLED THE JACCARD INDEX

- The second form is the one you compute: the union is never built
- Subtracting the intersection is not an optimisation — add the two areas and you have counted the overlap twice
- Range $[0, 1]$, and it is exactly 1 only when $A = B$

The two areas

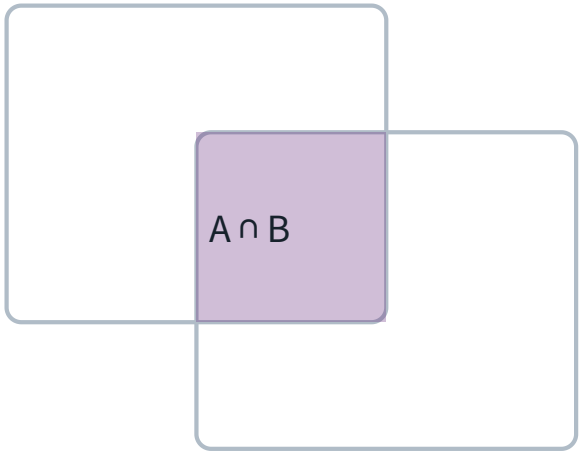
The two boxes

ground truth A



prediction B

Intersection



clamp each side at zero, or two negatives multiply to a positive

Union

= area(A) + area(B) − area(A ∩ B), so the overlap is not counted twice

The intersection of two axis-aligned boxes is another axis-aligned box, which is what makes this cheap.

Computing it, and the one line that matters

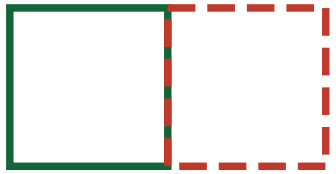
```python
def iou(a, b):
    """a, b are [x1, y1, x2, y2]."""
    lt = np.maximum(a[:2], b[:2])          # top-left of the overlap
    rb = np.minimum(a[2:], b[2:])          # bottom-right of the overlap
    wh = np.clip(rb - lt, 0.0, None)       # <- the whole lecture
    inter = wh[0] * wh[1]
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter)

assert iou(np.array([0, 0, 10, 10.]), np.array([0, 0, 10, 10.])) == 1.0
assert iou(np.array([0, 0, 10, 10.]), np.array([20, 20, 30, 30.])) == 0.0
```

Without the clamp, two disjoint boxes give a negative width and a negative height, whose product is a **positive** intersection. We come back to that in twenty minutes with a measurement.

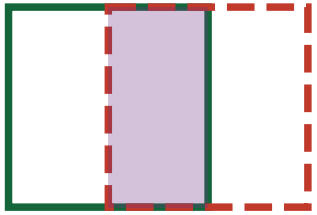
Four pairs, four values

300 px away
IoU = 0.000

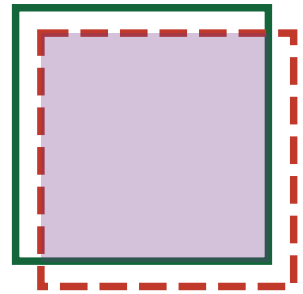
edge to edge
IoU = 0.000



half overlapping
IoU = 0.333



nearly identical
IoU = 0.681



✗ **The first two panels are very different pictures and the same number. Hold on to that.**

It meets all four requirements

Requirement	Why IoU satisfies it
1 when coincident	numerator equals denominator
✓ punishes too large	**✓ $\lvert B \rvert$ grows, so the denominator grows**
✓ punishes too small	**✓ the numerator shrinks with $\lvert A \cap B \rvert$**
dimensionless	area over area

✓ And the whole-image baseline scores 0.086 on our corpus — a box that is always right about *something* and never about anything in particular.

IoU does three different jobs

1. **Matching.** Decide whether a detection counts as the same object as an annotation — today, at a threshold
2. **Suppression.** Decide whether two detections are duplicates of each other — later this lecture
3. **Training.** Serve as the loss a regression head descends — and this is the one that breaks

The first two use IoU as a comparison. Only the third needs it to have a derivative, and that is where the derivation has its teeth.

IoU as a loss

$$\mathcal{L}_{\text{IoU}} = 1 - \text{IoU}(A, B)$$

B IS PREDICTED; A IS FIXED

Attractive, because it optimises the thing that is reported. The alternative — a squared error on the four coordinates — optimises something else and hopes.

So: take the derivative with respect to the predicted box and descend.

The experiment

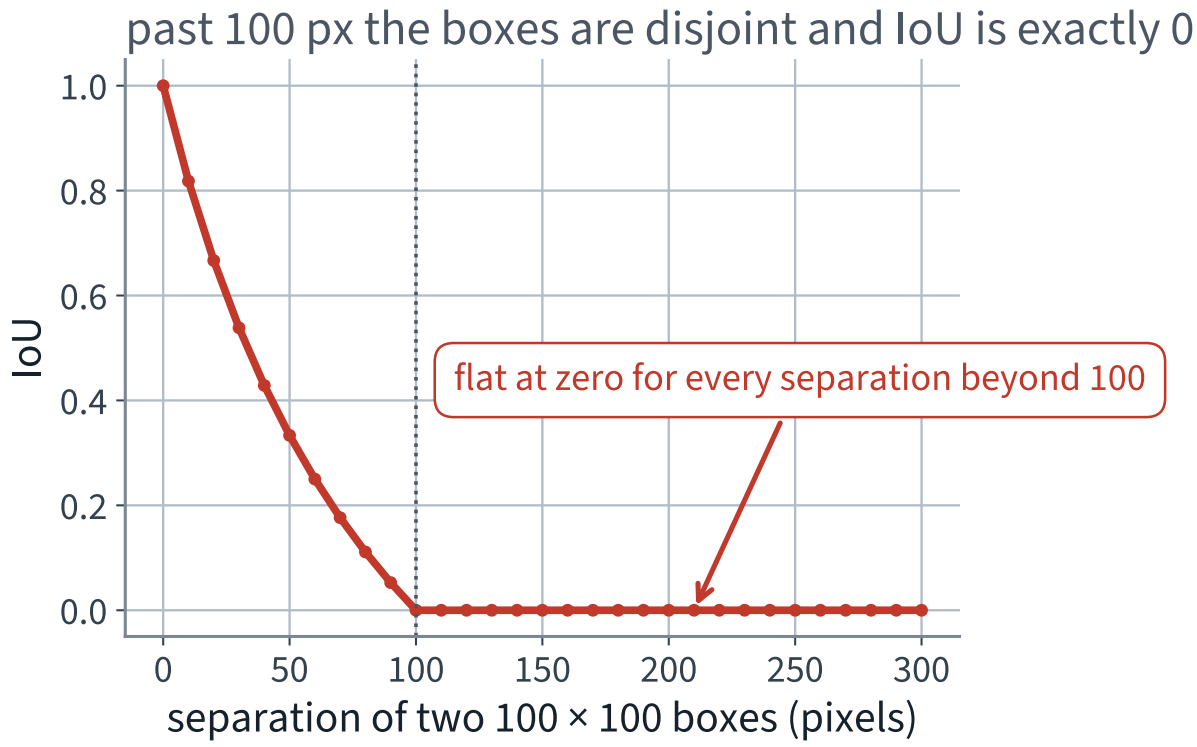
Two boxes, each 100×100. Fix the first at the origin. Slide the second to the right by d pixels and record the value and the gradient.

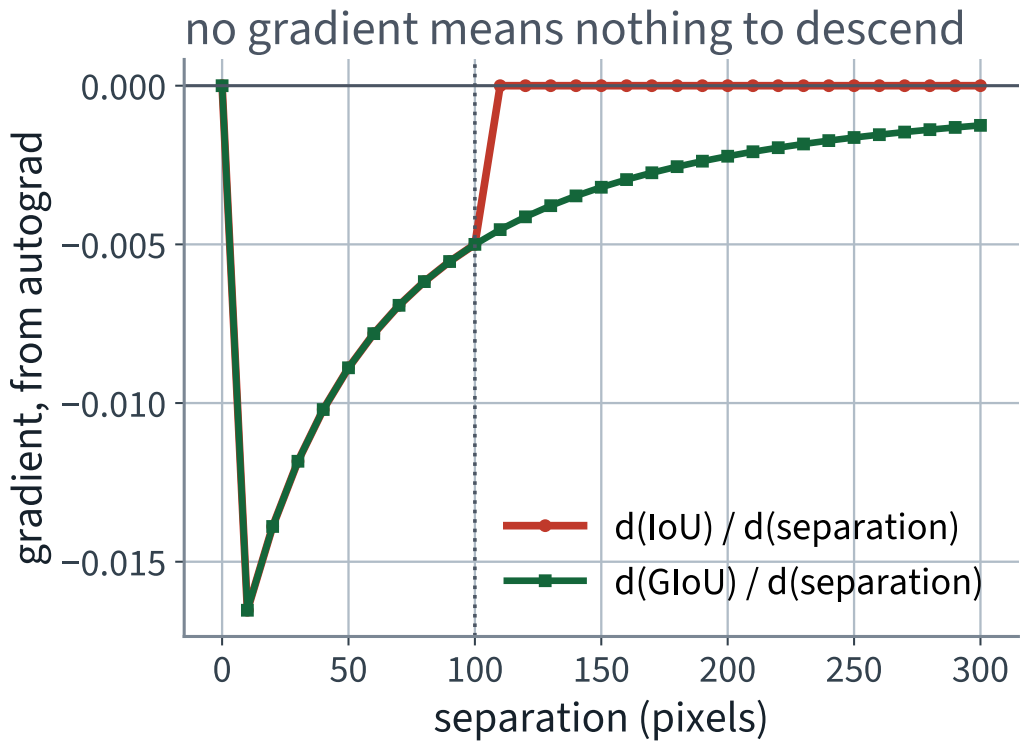
$$A = [0,\ 0,\ 100,\ 100], \qquad B(d) = [d,\ 0,\ d{+}100,\ 100]$$

SYNTHETIC BOXES — NO DATA, NO MODEL, NO NOISE

Synthetic on purpose. Nothing here depends on a dataset, and the conclusion is a property of the formula rather than of COCO.

Past 100 pixels there is nothing to descend





Twenty separations past 100 px, every one of them IoU $= 0$ and gradient exactly 0.

The same thing as numbers, from autograd

d	IoU	∂ IoU$/\partial d$	GIoU	∂ GIoU$/\partial d$
0	1.000	0.000	1.000	0.000
50	0.333	−0.0089	0.333	−0.0089
90	0.053	−0.0055	0.053	−0.0055
✗ 150	**✗ 0.000**	**✗ 0.000**	**✓ −0.200**	**✓ −0.0032**
✗ 300	**✗ 0.000**	**✗ 0.000**	**✓ −0.500**	**✓ −0.0013**

Rows four and five are the point. Two boxes 150 px apart and two boxes 300 px apart are, to IoU, exactly equally wrong.

Why the gradient is exactly zero, not merely small

For $d \geq 100$ the overlap width is $\max(0,\ 100 - d) = 0$, so

$$|A \cap B| \equiv 0 \quad \Longrightarrow \quad \mathrm{IoU} \equiv 0 \quad \Longrightarrow \quad \frac{\partial\, \mathrm{IoU}}{\partial d} \equiv 0$$

- The clamp makes the function *constant* on the whole disjoint region, not just small there
- A constant has no descent direction. Not a weak one — none
- This is a property of the formula. No optimiser, learning rate or initialisation repairs it

What that costs you in practice

THE FAILURE MODE

A proposal that starts anywhere outside the object it should find receives the same loss and the same zero gradient wherever it is. It cannot be moved towards the object, because nothing in the loss knows which way the object lies.

This is why detectors are not trained on IoU alone. It is also why the coordinate regression they *are* trained on needs anchors close enough to overlap in the first place.

FAILURE CONDITION — A LOSS THAT IS FLAT WHERE YOU NEED IT

IoU is exactly zero for every disjoint pair, so a proposal that starts outside the object gets the same loss and the same zero gradient wherever it starts. The optimiser is not slow here — it has no direction to follow at all.

The repair: put the empty space back in

Let C be the smallest axis-aligned box containing both A and B.

$$\mathrm{GIoU}(A, B) = \mathrm{IoU}(A, B) - \frac{|C| - |A \cup B|}{|C|}$$

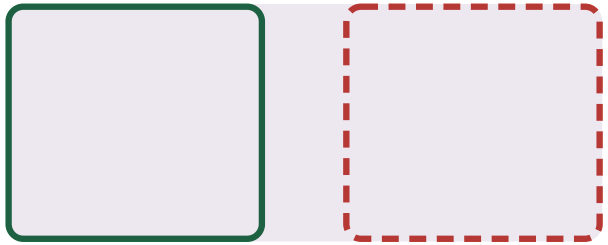
GENERALISED IOU

- The subtracted term is the fraction of the enclosing box that is *neither* prediction nor truth
- When the boxes are disjoint and move apart, $|C|$ grows and $|A \cup B|$ does not, so the penalty grows
- Range $(-1, 1]$, and it equals IoU whenever $C = A \cup B$

The enclosing box does the work

Disjoint, close

C, the smallest box containing both

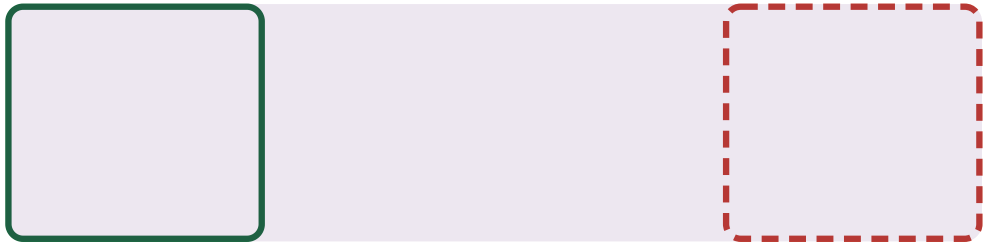


IoU = 0

GIoU = −0.14

the gap is small relative to C,
so the penalty is small

Disjoint, far



IoU = 0, still

GIoU = −0.48

C has grown, the gap has grown
faster, and the penalty follows

$$GIoU = IoU - \text{area}(C \setminus (A \cup B)) / \text{area}(C)$$

one term that keeps moving where IoU has stopped

Same IoU on both sides, different GIoU — and the difference points in the direction the box has to move.

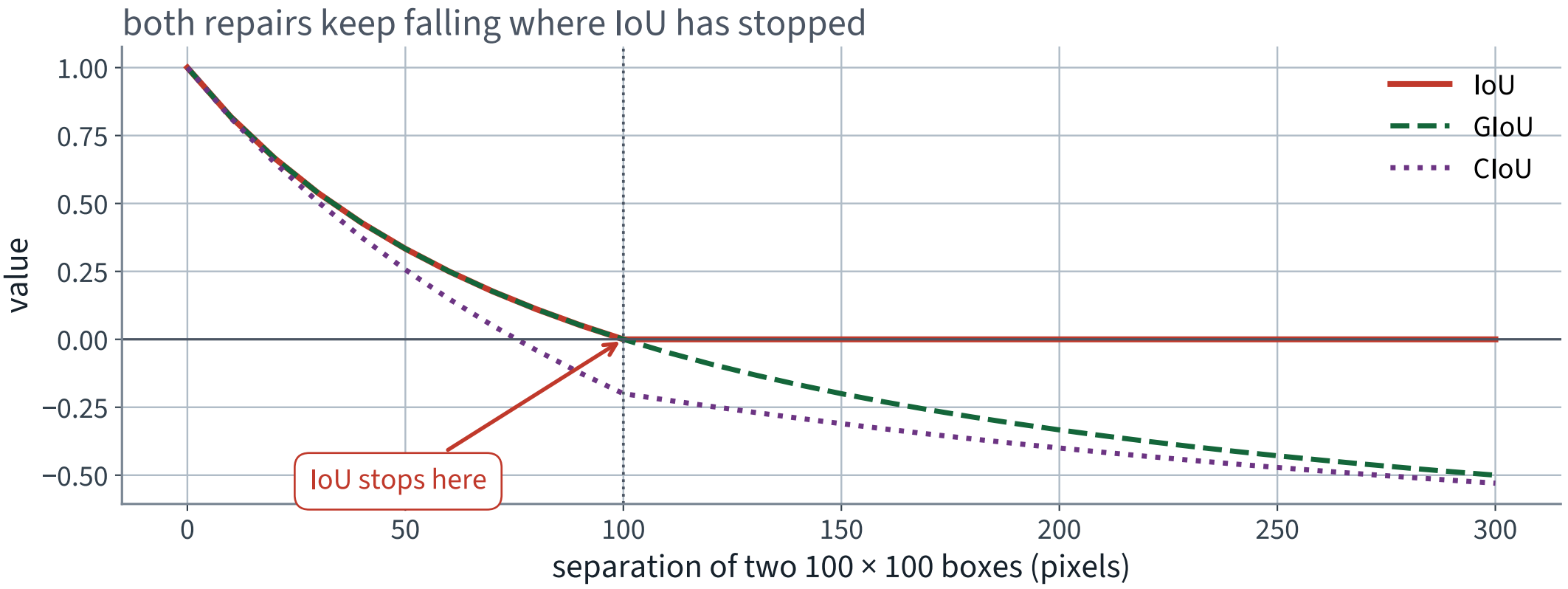
CIoU: two more terms, for two more faults

$$\mathrm{CIoU} = \mathrm{IoU} - \frac{\rho^2(\mathbf{c}_A, \mathbf{c}_B)}{\ell^2} - \alpha\, v$$

P CENTRE DISTANCE, ℓ DIAGONAL OF C

- The middle term is centre distance, normalised by the enclosing diagonal so it stays dimensionless
- v measures disagreement in aspect ratio; α weights it so that it matters more once the boxes already overlap
- It converges faster than GIoU because it moves the centre directly rather than shrinking a gap

Both repairs keep moving where IoU has stopped



Past 100 px, IoU is a horizontal line and the other two are still descending.

From one box to a whole detector

IoU scores one pair. A detector emits 4,419 boxes over 128 images, most of them wrong, all of them carrying a score.

- Fixing a score threshold gives one precision and one recall — and throws away everything the ranking knew
- Lecture 3 already solved this shape of problem

✓ A detector is a ranking. Score it as one.

Turning detections into true and false positives

1. Fix a class and an IoU threshold t
2. Sort every detection of that class, over the whole corpus, by score, highest first
3. Walk down the list. For each detection, find the best *unmatched* annotation in the same image
4. If that IoU is at least t: a true positive, and the annotation is now matched. Otherwise: a false positive

Step 3's word *unmatched* is what makes a second box on the same bottle a false positive rather than a second success.

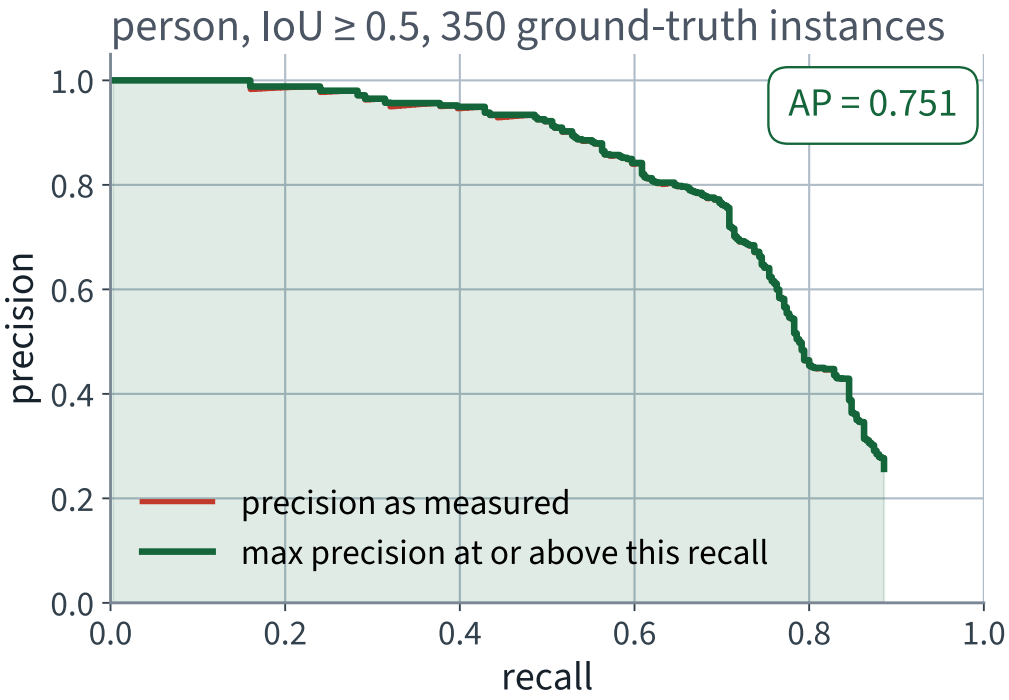
Precision and recall, unchanged from Lecture 3

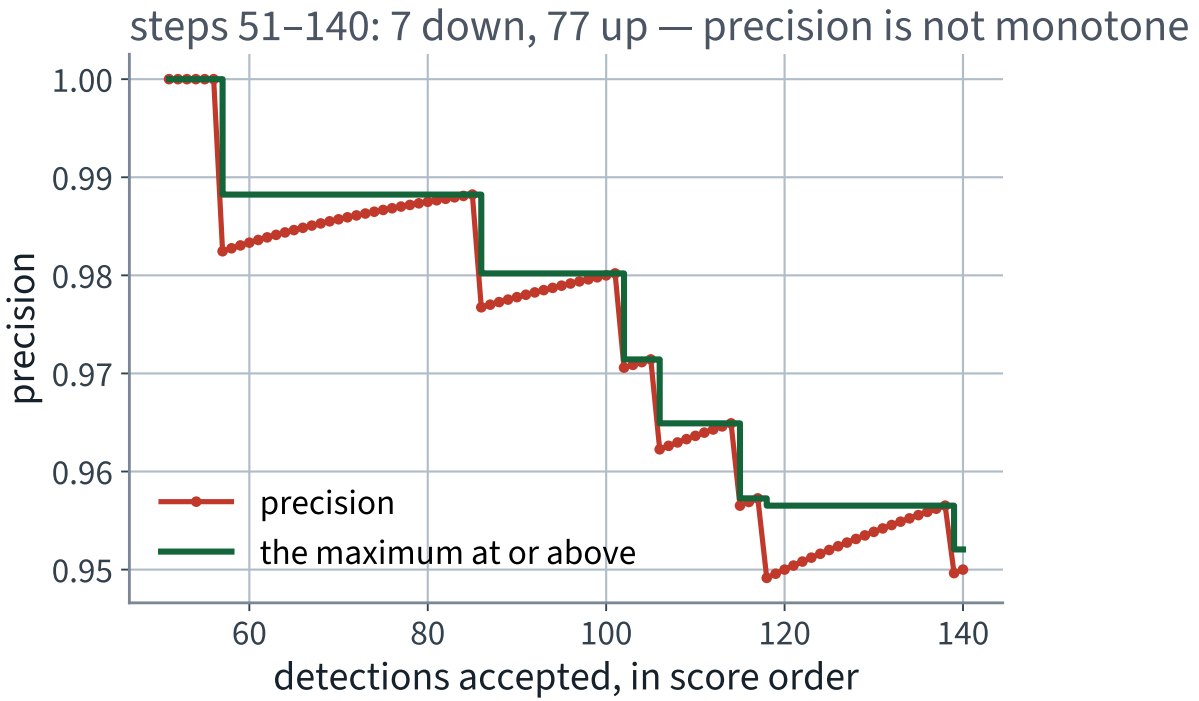
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{recall} = \frac{\text{TP}}{\text{annotations of this class}}$$

- Both are computed *cumulatively*, after each detection in the ranking — so a corpus of 1,209 person detections gives 1,209 points
- The recall denominator is fixed, so recall is non-decreasing as you go down the list
- Precision has both a numerator and a denominator moving

Precision is not monotone here either





The right-hand panel is Lecture 3's counterexample on boxes instead of digits: 7 steps down and 77 up, in 90 steps.

Every step accounted for, exactly

Walking the person ranking, one detection at a time	Steps
the detection is a false positive — precision falls	✗ 899
it is a true positive and precision was below 1 — precision rises	254 ✓
it is a true positive and precision was already 1 — no change	55

- 899 is exactly the number of false positives
- $254 + 55 = 309$, which is the 310 true positives less the one at the very top of the ranking, which has no step above it
- The 55 flat steps are one contiguous run: the top 56 person detections in the whole corpus are all correct

✓ This is not a tendency. It is an exact classification of all 1,208 steps, in the same form as Lecture 3.

The repair Lecture 3 promised you

> *From Lecture 3: "Average precision is defined using the maximum precision at or above each recall level. That maximum is there for exactly one reason: to replace a non-monotone quantity by a monotone one. Lecture 14 builds mAP on top of it."*

$$p_{\text{env}}(r) \;=\; \max_{\tilde{r} \geq r} p(\tilde{r})$$

THE ENVELOPE — NON-INCREASING BY CONSTRUCTION

Why a maximum, and not a smoothing

- It has an operational reading: *if you are willing to accept recall r, here is the best precision available to you at that recall or better*
- It is monotone by construction, so the area under it is well defined and does not depend on where you sampled
- It is one line of code and has no parameters — nothing to tune, nothing to report

```
env = precision.copy()
for i in range(len(env) - 2, -1, -1):
    env[i] = max(env[i], env[i + 1])       # sweep right to left
```

Average precision

$$\mathrm{AP} \;=\; \int_0^1 p_{\mathrm{env}}(r)\,\mathrm{d}r \;=\; \sum_k \left(r_k - r_{k-1}\right) p_{\mathrm{env}}(r_k)$$

AREA UNDER THE ENVELOPED CURVE

- The sum only has terms where recall actually changed, which is exactly at the true positives
- **No threshold appears anywhere.** That is the whole point: the number does not depend on a cut-off nobody chose

Average precision for one class, on 128 images

person, IoU ≥ 0.5	Value
Annotated instances	350
Detections in the ranking	1,209
True positives	310
Highest recall reached	0.886
✓ AP	**✓ 0.751**

Recall stops at 0.886 because 40 annotated people are never detected at all, at any score. No threshold can recover them, and AP is honest about that.

The first mean: over classes

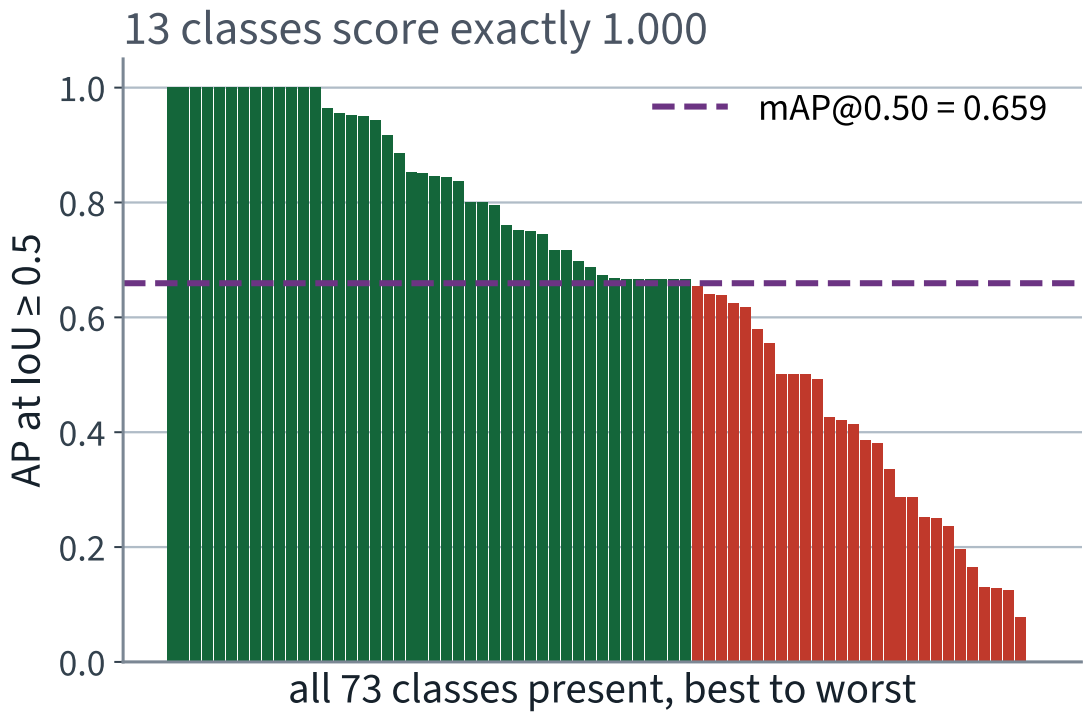
$$\mathrm{mAP}(t) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \mathrm{AP}_c(t)$$

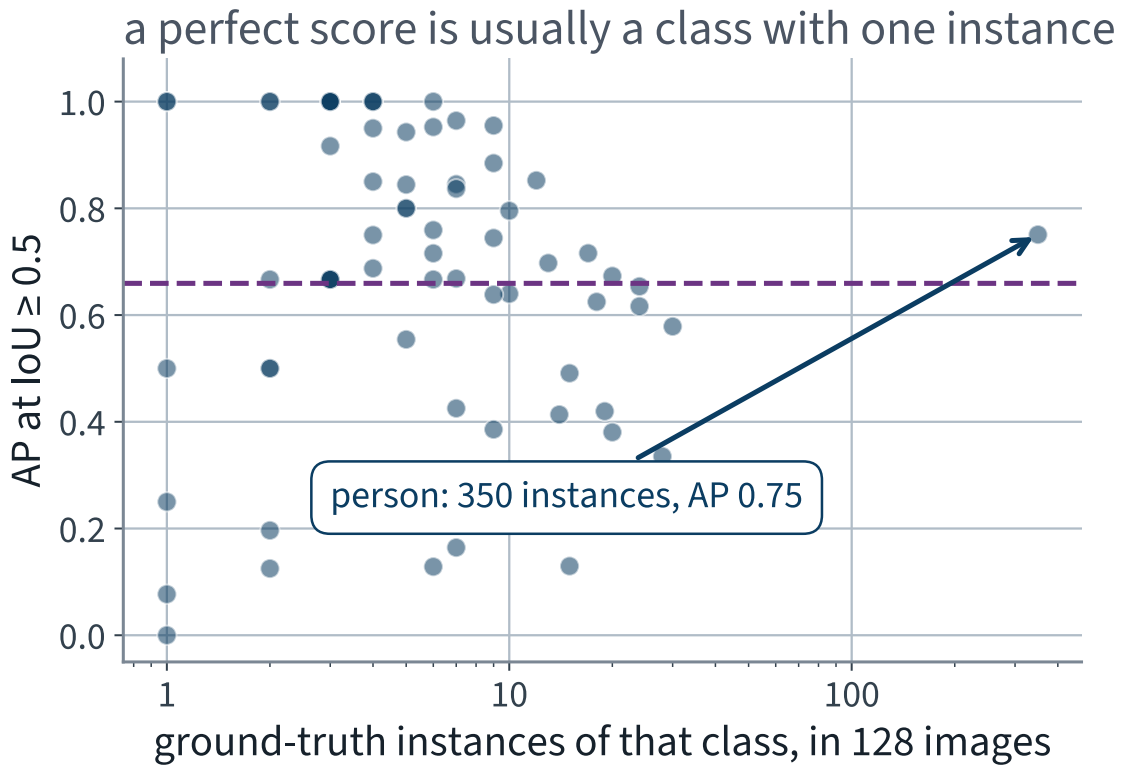
$\mathcal{C}$ = THE CLASSES PRESENT IN THE CORPUS

- Unweighted. A class with one annotation counts as much as `person` with 350
- We average over the 73 classes that appear in our 128 images. The other 7 of COCO's 80 have no annotations here and no AP to average

✗ That choice — which classes are in the mean — is a decision, it changes the number, and papers do not always say which one they made.

What the first mean hides





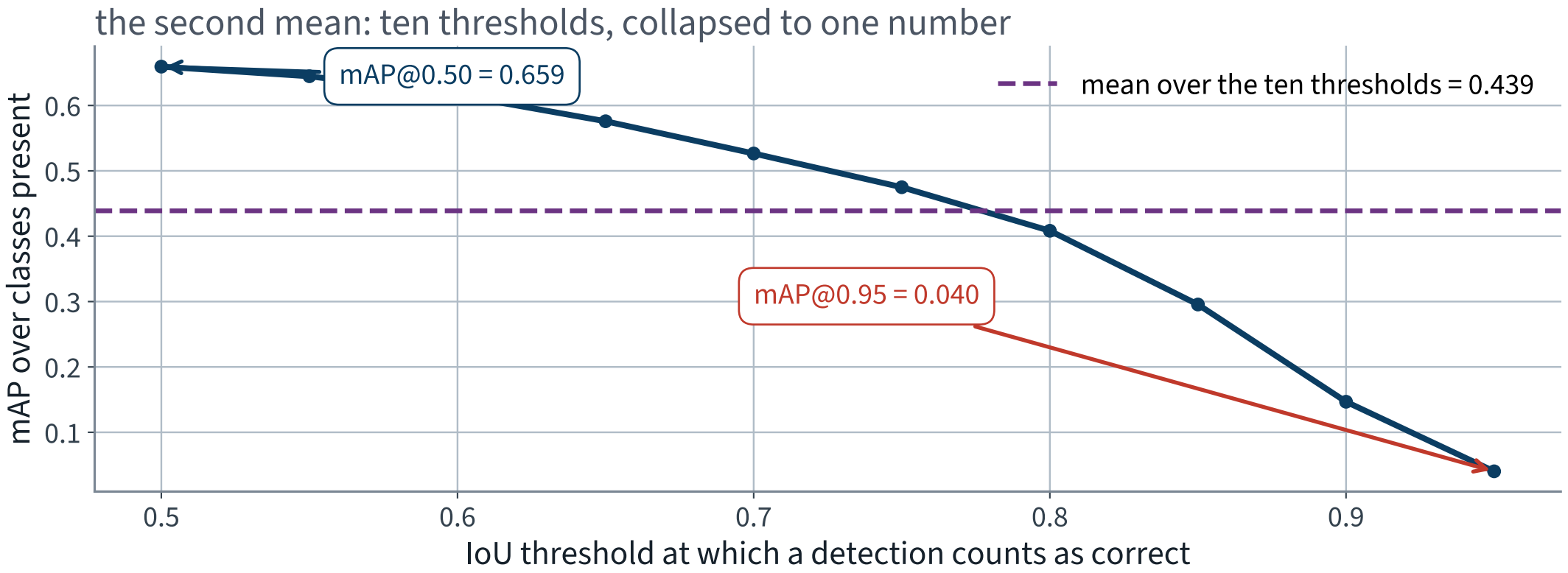
Thirteen classes score exactly 1.000, and they are the ones with one or two instances in the whole corpus.

The second mean: over IoU thresholds

$$\mathrm{mAP}_{[.5:.95]} = \frac{1}{10} \sum_{t \in \{0.50,\, 0.55,\, \ldots,\, 0.95\}} \mathrm{mAP}(t)$$

- At $t = 0.50$ a box that half-covers the object counts as correct
- At $t = 0.95$ it must be very nearly exact
- Averaging the ten is COCO's headline number, and it is what "mAP" means unqualified in a modern paper

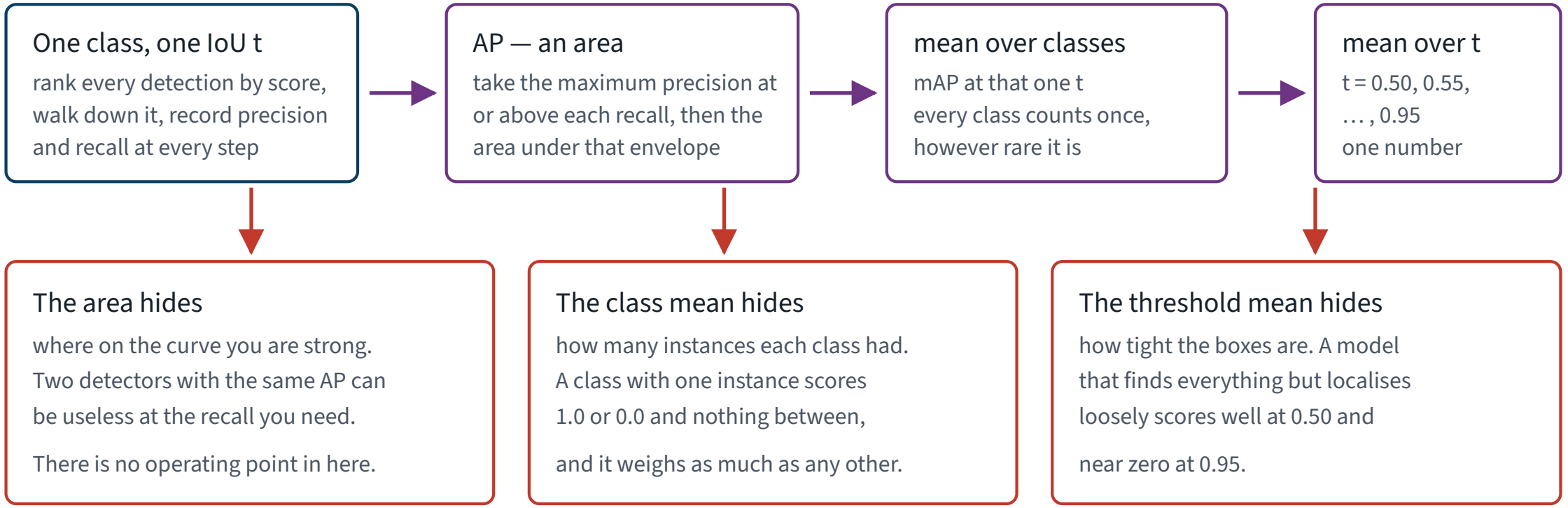
What the second mean hides



0.659 at the loosest threshold and 0.040 at the tightest. One number in the middle stands for both.

A mean of a mean of an area

mAP is a mean of a mean of an area



Three collapses, one number. Each collapse throws away a different thing, and none of them is recoverable from the result.

Our detector, on 128 images

Metric	Value
mAP at IoU 0.50	0.659
mAP at IoU 0.75	0.475
✓ mAP averaged over 0.50 to 0.95	**✓ 0.439**

Every one of these is a measurement on **128** images and 73 classes, and is quoted with that attached.

the derivation · Summary

- IoU is intersection over union: dimensionless, 1 for identical boxes, and it punishes a box for being too large or too small
- It is **constant at zero** for every disjoint pair, so its gradient there is exactly zero and there is nothing to descend
- GIoU adds the empty part of the enclosing box; CIoU adds centre distance and aspect ratio. Both keep a gradient where IoU has none
- Average precision is the area under the *maximum* precision at or above each recall — the maximum exists to repair the non-monotonicity proved in Lecture 3
- mAP averages AP over classes and then over ten IoU thresholds. It is a mean of a mean of an area, and each level hides something different

THE METHOD

Running a detector, and reading it

30 minutes · examinable

The notebook for this lecture

Open Lecture 14 in Colab

- **Runs on free CPU.** 128 COCO images and a pretrained detector, inference only — nothing is trained here
- IoU implemented and checked against the cases the derivation names, including the disjoint one where the gradient vanishes
- AP computed from the sawtooth and from its running maximum, so the repair is something you compute rather than accept

> **WHAT TO CHANGE**
>
> Move one predicted box further and further from its match, and watch IoU sit at exactly zero the whole way. That flat zero is the vanishing gradient the derivation predicts.

Step 1 · Specify, before anything is generated

Input	128 JPEG files and one COCO annotation file
Output	per image: boxes in corner form, category ids, scores; plus one integer count
Constraint	nothing is trained; the pretrained weights are used as they ship
Check	every predicted box lies inside the image; scores are non-increasing; the count equals the number of boxes kept

The *Check* row is the one people skip, and it is the one that catches a corner-versus-size mix-up in one line.

Step 2 · Load the corpus, and convert the boxes once

```
1  import json, numpy as np
2
3  raw = json.loads(ANN_PATH.read_text())
4  images = sorted(raw["images"], key=lambda i: i["id"])[:128]
5  ids = {im["id"] for im in images}
6
7  gt = {i: {"boxes": [], "labels": []} for i in ids}
8  n_crowd = 0
9  for a in raw["annotations"]:
10     if a["image_id"] not in ids:
11         continue
12     if a["iscrowd"]:                       # a refusal to decide
13         n_crowd += 1
14         continue
15     x, y, w, h = a["bbox"]                 # COCO gives x, y, w, h
16     gt[a["image_id"]]["boxes"].append([x, y, x + w, y + h])
17     gt[a["image_id"]]["labels"].append(a["category_id"])
```

The conversion happens **once**, at the edge of the program. From line 17 onward every box in memory is corners.

Step 3 · Assert it, do not hope

```
for iid, g in gt.items():
    b = np.asarray(g["boxes"], dtype=float).reshape(-1, 4)
    assert (b[:, 2] >= b[:, 0]).all(), "x2 < x1 — you read w as x2"
    assert (b[:, 3] >= b[:, 1]).all(), "y2 < y1 — same bug"
    gt[iid]["boxes"] = b

total = sum(len(g["boxes"]) for g in gt.values())
assert len(gt) == 128
assert total == 898, total
print(f"{len(gt)} images, {total} objects, {n_crowd} crowd regions dropped")
```

Reading `w` as `x2` gives `x2 < x1` for every box wider than its own left edge, which is almost all of them. Two lines catch it; nothing else will.

Step 4 · The detector, off the shelf

```
import torch
from torchvision.models.detection import (
    fasterrcnn_resnet50_fpn, FasterRCNN_ResNet50_FPN_Weights)

weights = FasterRCNN_ResNet50_FPN_Weights.COCO_V1
model = fasterrcnn_resnet50_fpn(weights=weights)
model.eval().to(DEVICE)                    # eval(), every time
preprocess = weights.transforms()

print(len(weights.meta["categories"]), "label slots")
print(weights.meta["_metrics"])
```

`model.eval()` is in the silent-failure catalogue from Lecture 10. Here it changes the batch-norm statistics inside a frozen ResNet, and nothing raises.

Non-examinable: the torchvision weights API.

Step 5 · Run it over the corpus

```python
from PIL import Image

preds = {}
with torch.inference_mode():                # no graph, no gradients
    for im in images:
        img = Image.open(IMG_DIR / im["file_name"]).convert("RGB")
        out = model([preprocess(img).to(DEVICE)])[0]
        preds[im["id"]] = {k: v.cpu().numpy() for k, v in out.items()}

assert len(preds) == 128
```

⏱ **about 40 seconds** on an Apple Silicon laptop with the MPS backend — 0.31 s per image, measured on the machine that made these slides. On a CPU-only runtime expect several minutes; no output does not mean it has hung.

Step 6 · Read the shape before you read the answer

```python
p = preds[images[0]["id"]]
for k, v in p.items():
    print(f"{k:8s} {v.shape} {v.dtype}")

# boxes    (88, 4) float32
# labels   (88,)   int64
# scores   (88,)   float32
assert np.all(np.diff(p["scores"]) <= 0), "not sorted by score"
```

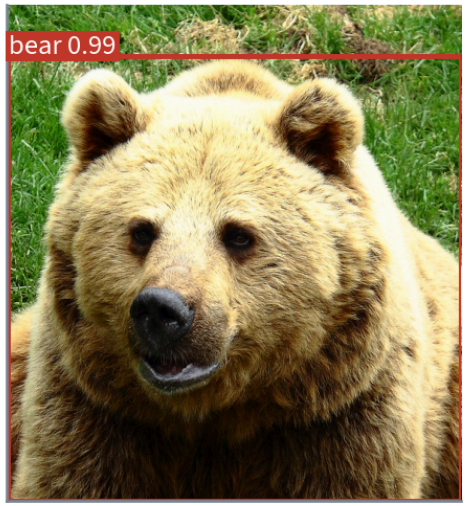
- Eighty-eight boxes for one photograph. Not eighty-eight objects
- The three arrays share an index: row k of `boxes` goes with `labels[k]` and `scores[k]`
- Sorted by score, descending — assert it rather than assume it

It works, and you can see that it works

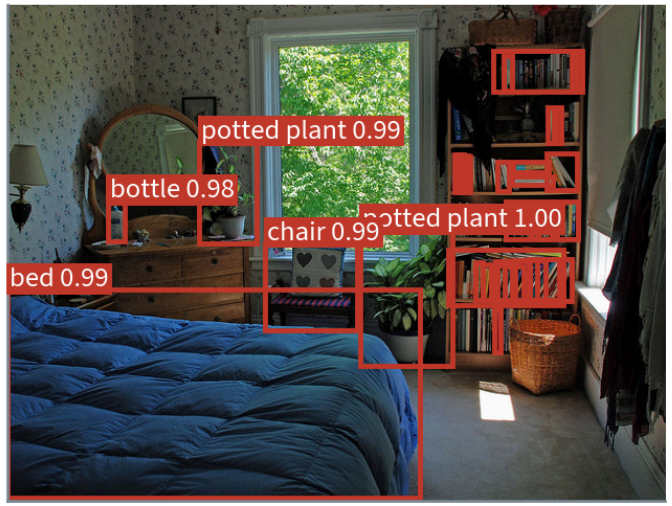
Faster R-CNN, score ≥ 0.5, on three of our 128 images; labels shown for scores ≥ 0.90

25 boxes (truth: 20)

1 box (truth: 1)



31 boxes (truth: 17)

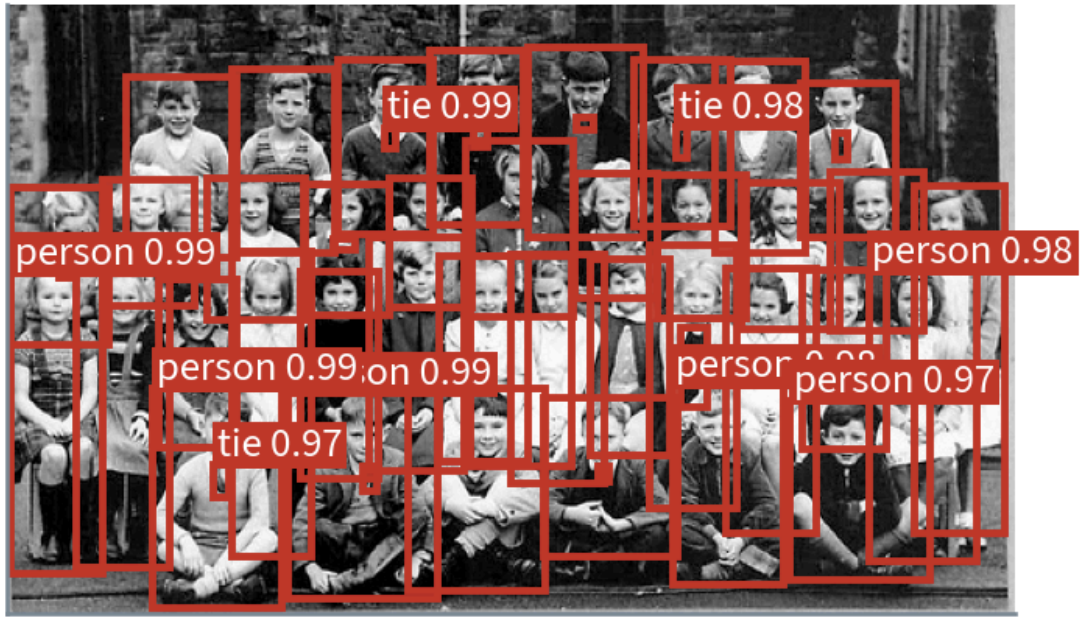


The bear is right. The two rooms have more boxes than there are objects, and you can see which ones are wrong without being able to say how wrong.

Where it visibly struggles

the two most crowded images in the first forty; labels shown for scores ≥ 0.97

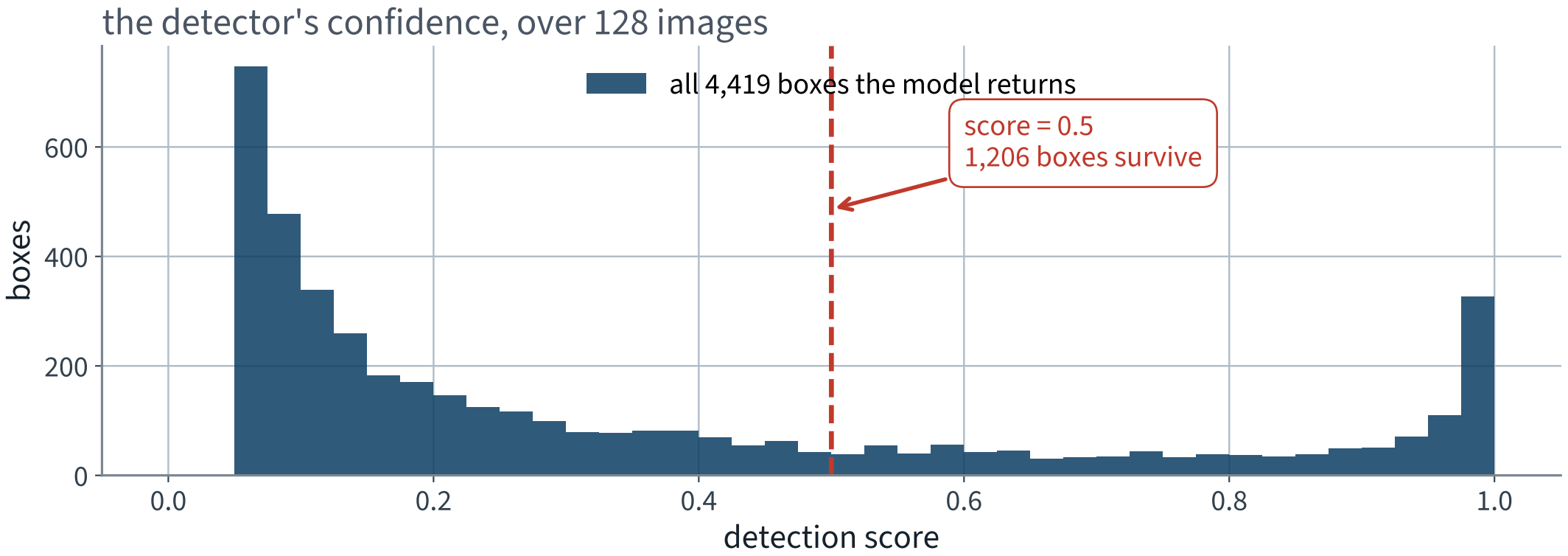
50 boxes (truth: 22)



25 boxes (truth: 20)

Fifty boxes for twenty-two objects. Several children carry two or three, and nothing says which is the good one.

Step 8 · Where is all that confidence?



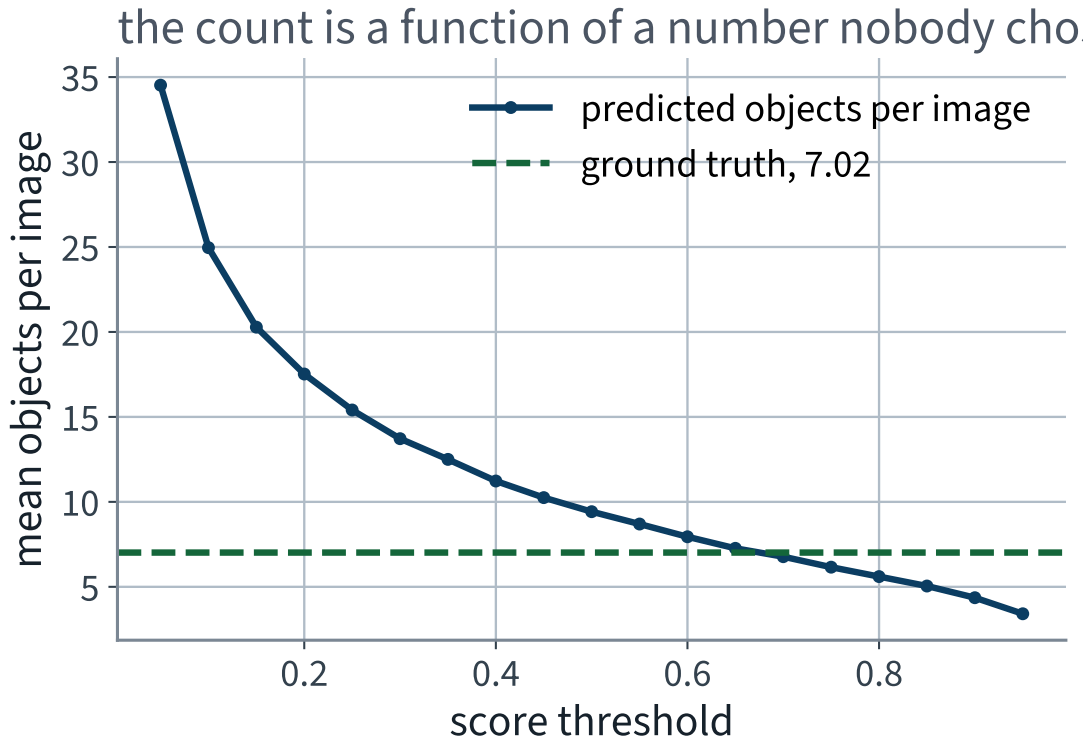
The model returns 4,419 boxes over 128 images and is not confident about most of them.

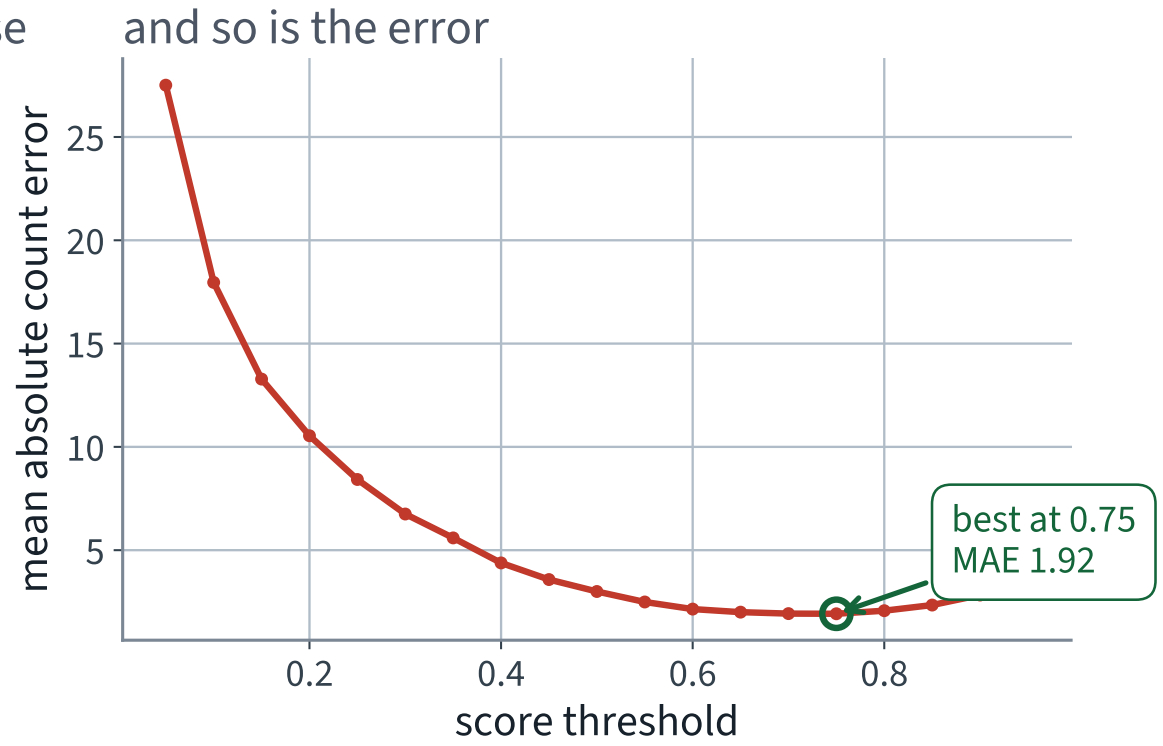
Step 9 · There is already a cut-off, and it is not yours

- `box_score_thresh` defaults to **0.05**: anything below is never returned at all
- `box_nms_thresh` defaults to **0.5**: near duplicates are already removed before you see them
- `box_detections_per_img` defaults to **100**

> **REVIEWER QUESTION 5**
>
> *What is the default I did not ask for?* Three of them, and one is a hard cap you will hit: our busiest image returns exactly 100 boxes.

Step 10 · The count is a function of the threshold





Nothing in the model chose 0.5. Move the cut-off and the answer to the stakeholder's question changes by a factor of ten.

Pick one, and write down why

> **OUR OPERATING POINT**
>
> **0.5**, because it is the conventional reporting threshold and because it was chosen *before* looking at the error curve.

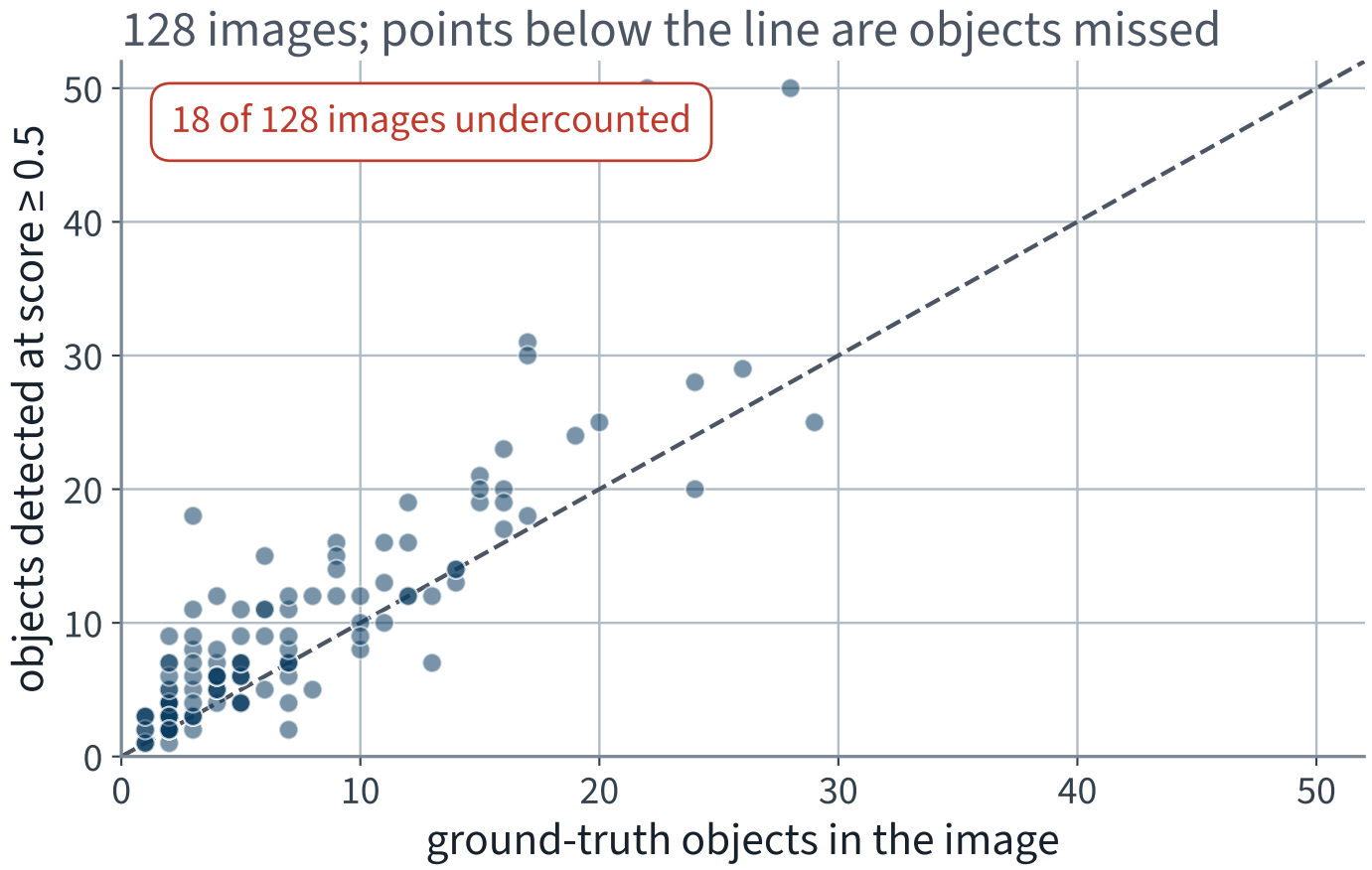
The minimum of the right-hand panel is at 0.75, with MAE 1.92. Adopting *that* would be choosing a hyperparameter on the same data you then report — Lecture 2's error, in a new costume. We will quote it as a diagnostic and not as a result.

Step 11 · Count the objects

```
THRESH = 0.5

n_pred = np.array([int((preds[im["id"]]["scores"] >= THRESH).sum())
                   for im in images])
n_true = np.array([len(gt[im["id"]]["labels"]) for im in images])

assert n_pred.shape == n_true.shape == (128,)
mae  = np.abs(n_pred - n_true).mean()
bias = (n_pred - n_true).mean()
print(f"count MAE  {mae:.2f}")
print(f"signed     {bias:+.2f}   (positive = too many boxes)")
```

Two numbers, not one. The absolute error says how far off; the signed error says which way.

Step 12 · Every image, plotted



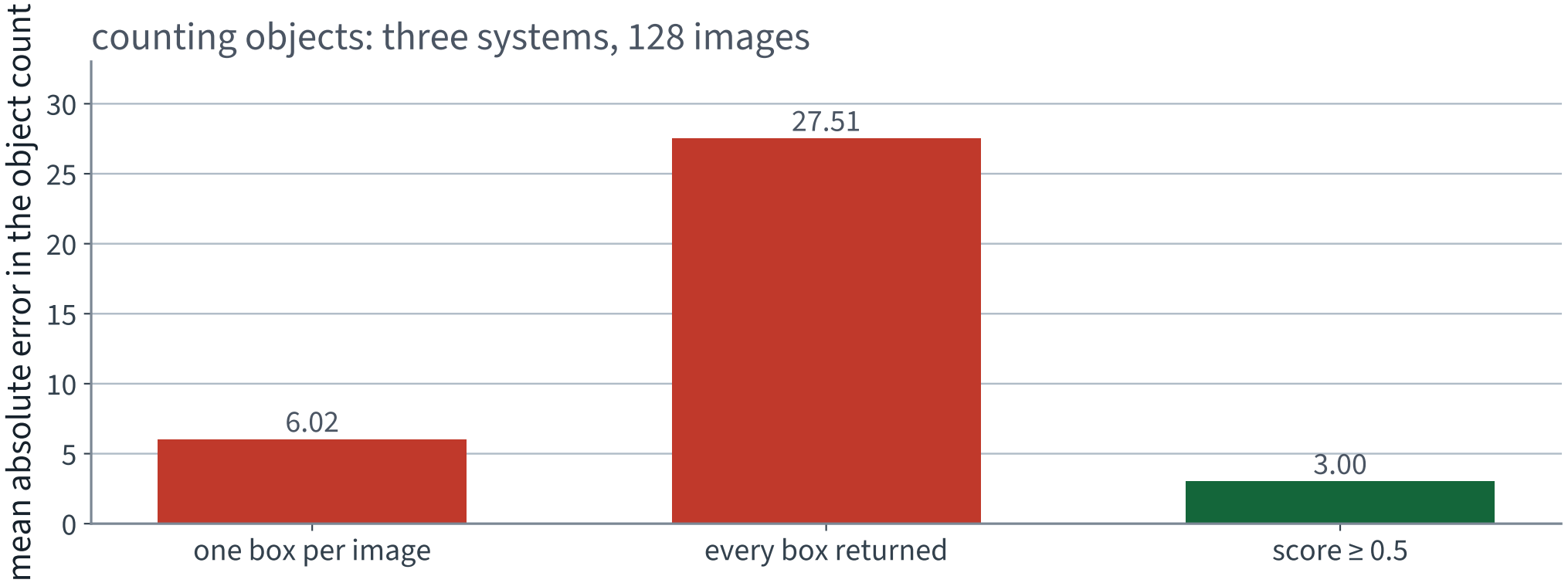
The cloud sits *above* the diagonal: the failure mode is too many boxes, not too few.

The result, on 128 images

At score ≥ 0.5	Value
Count MAE	**3.00 ✓**
Signed count error	**✗ +2.41**
Images counted exactly right	27 of 128
Images with too many boxes	**✗ 83**
Images with too few	18
Boxes kept in total	1,206

✓ 3.00 against a baseline of 6.02. The model halves the error of a system that never looks at the image.

Three systems, one metric



The middle bar is the same model with one line deleted, and it is four times worse than predicting nothing at all.

Before quoting our own number, check it against someone else's

We derived AP on these slides and implemented it ourselves. That is exactly the situation in which a result should not be believed.

mAP on our 128 images	Ours	Reference evaluator
at IoU 0.50	0.659	0.660
at IoU 0.75	0.475	0.476
averaged 0.50 to 0.95	0.439	0.440

✓ Largest disagreement: 0.0014. The reference is the official COCO evaluator, run on the same predictions and the same annotations, by a library we did not write.

Non-examinable: the reference library. Examinable: that you check an implementation against one.

Now compare 128 images against 5,000

mAP, averaged 0.50 to 0.95	Value	Images
Ours	43.9	128
Reported by torchvision for these weights	37.0	5,000

✗ Our number is 6.9 points higher, and the model is identical.

Nothing about the detector changed. The difference is the sample: 128 images, of which many are easy, and 73 classes several of which have a single instance that happened to be found. This is what a small evaluation set does, and it does it in the flattering direction more often than not.

How many boxes are there really?

231.7

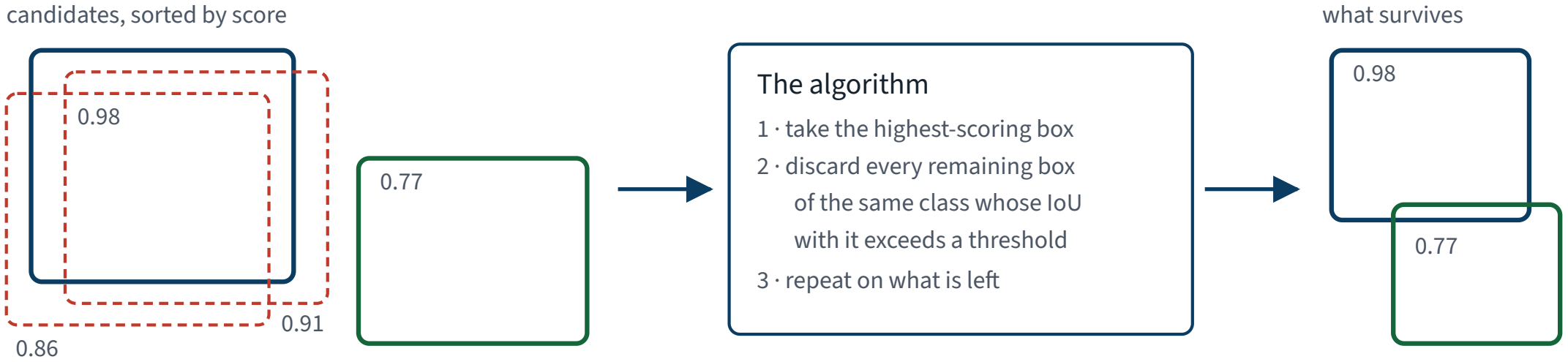
candidate boxes per image, with suppression switched off

against 23.6 after suppression at IoU 0.5, and 7.02 actual objects

The detector you ran earlier today had already thrown away nine tenths of its own output before you saw it, using IoU, at a threshold you did not set.

The rule

Non-maximum suppression, greedily



The cost, stated plainly

Two genuinely different objects that overlap heavily — one person in front of another — are one box after suppression.
The threshold buys precision with recall, and it is a number nobody in the room chose.

Greedy, per class, and it deletes information that cannot be recovered afterwards.

Suppression, in code

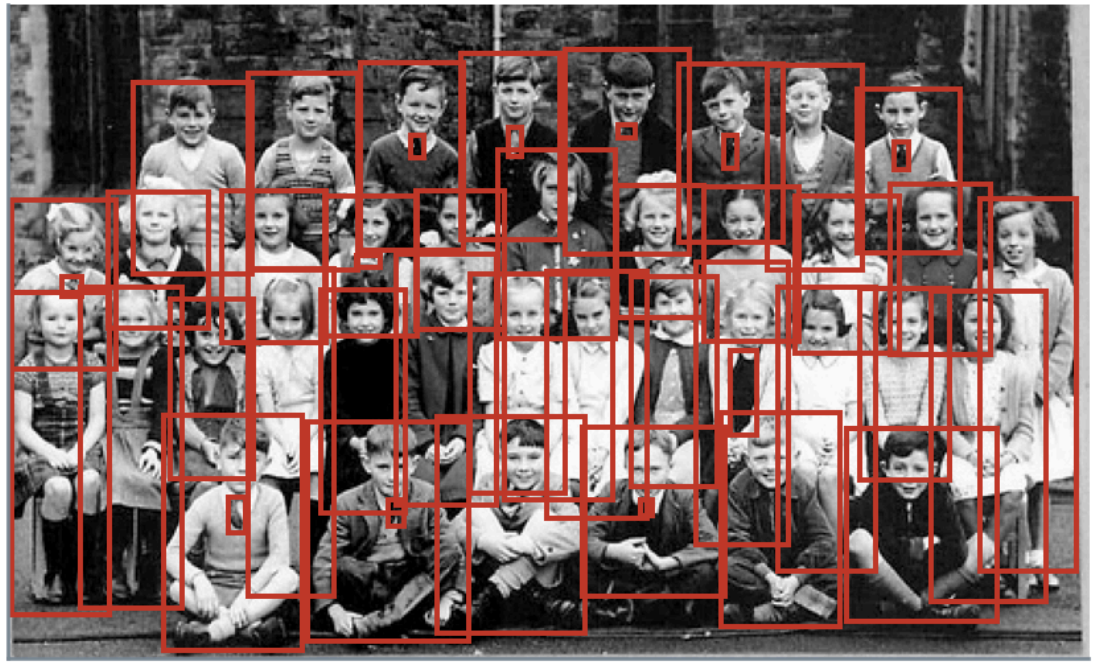
```
from torchvision.ops import batched_nms

keep = batched_nms(torch.tensor(boxes),      # (N, 4), corners
                   torch.tensor(scores),     # (N,)
                   torch.tensor(labels),     # (N,) — per class
                   iou_threshold=0.5)

assert len(keep) <= len(boxes), "suppression cannot add boxes"
boxes, scores, labels = boxes[keep], scores[keep], labels[keep]
```

`batched_nms` rather than `nms`: a bottle overlapping a glass must not suppress it, because they are different classes and both are true.

Before and after, on one image

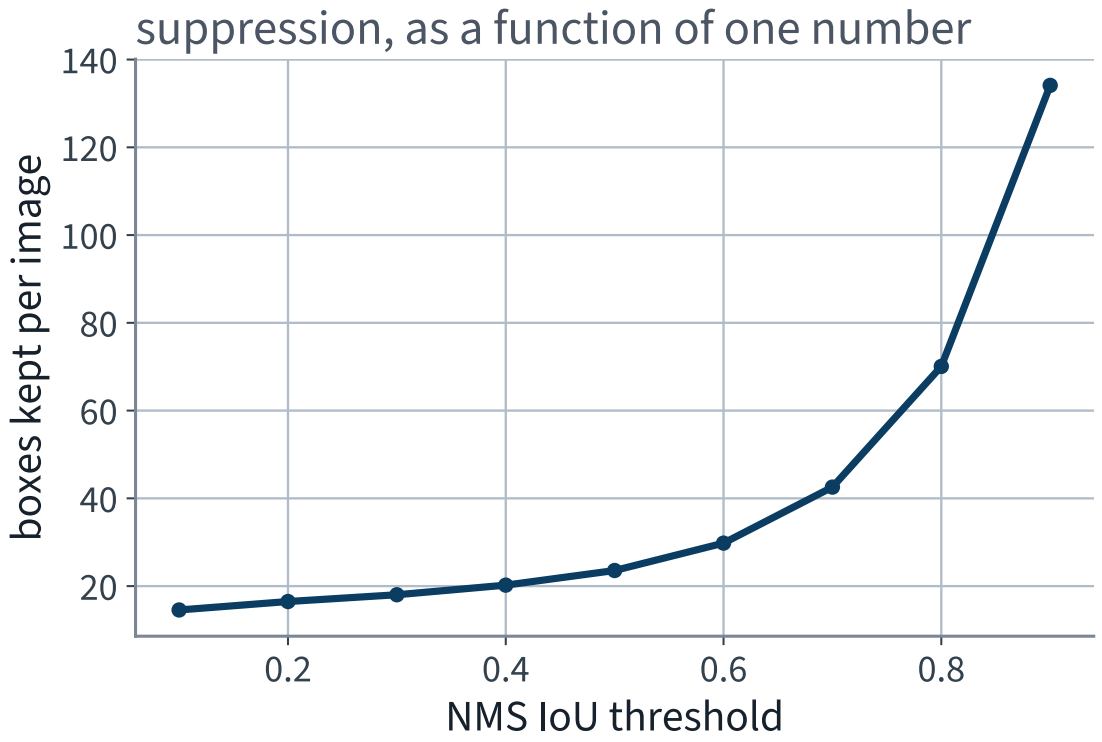
no suppression: 300 boxes (the 300-candidate cap)

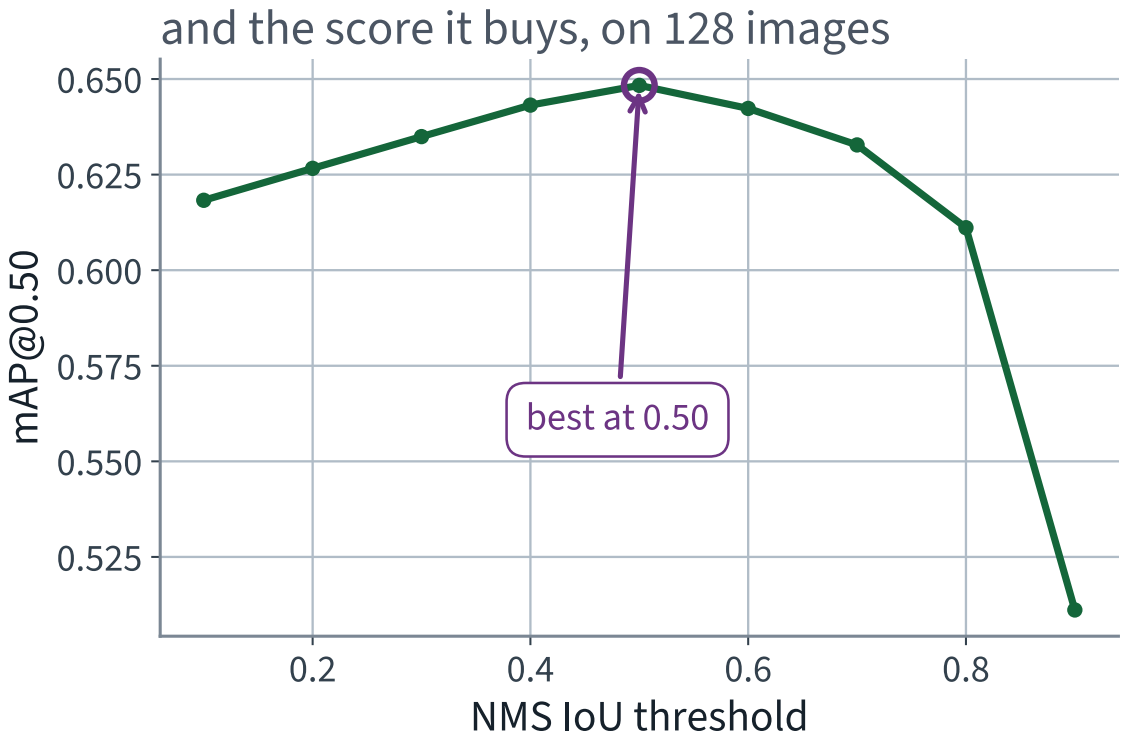
NMS at IoU 0.5: 50 boxes



Same model, same image, same score threshold. The only difference is one IoU comparison between pairs of boxes.

And it is another knob





The default of 0.5 happens to be near the best value on our corpus. That is a fact about this corpus, not a law.

THE LAST FIFTEEN MINUTES

Segmentation, and tracking

15 minutes

Detection on video is not tracking

Run the detector on every frame and you have boxes. You do not have *the same bottle* in frame 2 as in frame 1.

- Tracking asks for an identity that persists across frames
- Which turns each frame into an assignment problem: match this frame's detections to the existing tracks
- And the cheapest similarity between a track's predicted box and a new detection is — IoU

✓ the derivation pays for itself a third time.

A tracker, in four lines of description

1. Predict where each existing track's box will be in this frame
2. Compute IoU between every prediction and every new detection
3. Solve the assignment: pair them up to maximise total IoU
4. Unmatched detections start new tracks; unmatched tracks age and die

> **WHERE IT FAILS, AND WHY IT IS IOU'S FAULT AGAIN**
>
> An object that is occluded for ten frames reappears somewhere its predicted box does not overlap. IoU is zero, the assignment finds nothing, and the object is issued a new identity. The same flatness, in a new place.

EXTENSION THREE

Per-pixel prediction

15 minutes

A box was always an approximation

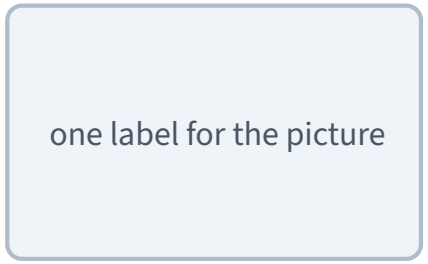
- A bicycle’s box is mostly not bicycle
- A person sitting cross-legged fills perhaps half of theirs
- For a picking arm, “somewhere in this rectangle” may not be good enough

Ask instead for a label on every pixel.

Four questions, four output shapes

Output shape, and what it can answer

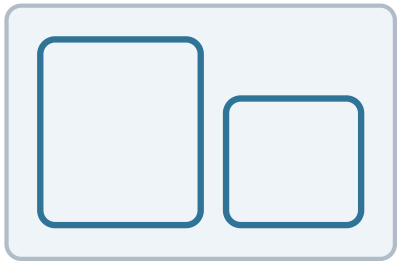
Classification



output: K numbers

Lectures 12–13

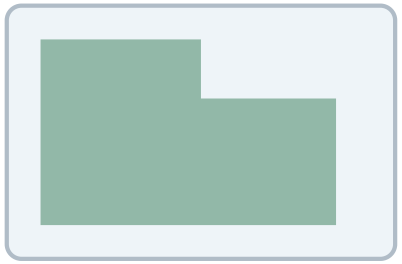
Detection



output: N boxes, N labels

Lecture 14

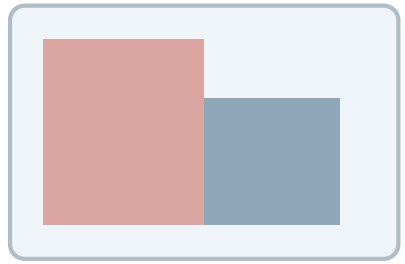
Semantic segmentation



output: one label per pixel

two people are one region

Instance segmentation



output: one mask per object

two people are two regions

Pooling throws away position, which is what makes it good for the first column and harmful for the last two — the equivariance-versus-invariance distinction from Lecture 12, now with a consequence you can see.

Semantic and instance segmentation differ in exactly one place: whether two touching objects of the same class are one region or two.

Semantic segmentation

- Output: one class label per pixel. Shape $H \times W$, not $N \times 4$
- Two people standing together are one `person` region. You cannot count them
- The architectural problem is that pooling destroyed the resolution the answer needs — which is Lecture 12's invariance, arriving as a cost
- The repair is upsampling with skip connections: transposed convolutions that recover spatial detail from the earlier, higher-resolution layers

Instance segmentation

- Output: one binary mask per detected object, plus its class and score
- Mask R-CNN is Faster R-CNN with one extra head that predicts a small mask inside each proposed box
- Two lines of code away from what you already ran

```
from torchvision.models.detection import (
    maskrcnn_resnet50_fpn, MaskRCNN_ResNet50_FPN_Weights)

model = maskrcnn_resnet50_fpn(
    weights=MaskRCNN_ResNet50_FPN_Weights.COCO_V1).eval().to(DEVICE)
out = model([preprocess(img).to(DEVICE)])[0]
print(out["masks"].shape)      # (N, 1, H, W), soft masks in [0, 1]
```

The same image, both ways

the image

semantic: 2 classes, one colour each

instance: 45 objects, one colour each

Middle: two classes, two colours. Right: 45 objects, 45 colours. Only the right-hand picture can answer “how many?”

Scoring a mask is the same argument again

- Mask AP replaces box IoU with **mask IoU**: intersecting pixel sets instead of rectangles
- Everything above — the ranking, the envelope, the area, the two means — is unchanged
- torchvision reports 34.6 mask mAP for these weights on the full 5,000, against 37.9 box mAP

A mask is harder than a box, which is what those two numbers say. We do not measure mask AP on our corpus; the pixel-level annotations are in the file and the code is an exercise.

Specimen exam question — Part A

> A predicted box and a ground-truth box are both 100×100 pixels and do not overlap.
>
> **1.** State the IoU, and its derivative with respect to the horizontal position of the predicted box.
> **2.** Explain in one sentence why this makes IoU unusable as a training loss on its own.
> **3.** Name one modification that repairs it and state which quantity it adds.

Four marks. Worked answer on the next slide.

Specimen answer — Part A

- **1.** IoU $= 0$; the derivative is 0. The intersection is clamped at zero over the whole disjoint region, so IoU is *constant* there
- **2.** A constant loss provides no descent direction, so gradient descent cannot move the predicted box towards the target from any starting point that does not already overlap it
- **3.** GIoU, which subtracts the fraction of the smallest enclosing box occupied by neither box; or CIoU, which additionally subtracts a normalised centre distance and an aspect-ratio term

✗ The common wrong answer to part 1 is "the gradient is very small". It is not small. It is zero, and the distinction is the entire question.

Specimen exam question — Part B

> A detector is evaluated on a corpus in which 30 of the classes present have exactly one annotated instance each. Its reported mAP at IoU 0.5 is 0.72.
>
> 1. State two distinct reasons why this number may be optimistic.
> 2. State one thing you would report alongside it.

Three marks.

Specimen answer — Part B

- **1a.** A class with one instance has AP either 1 or 0, so 30 near-coin-flips are averaged in with the same weight as a class with hundreds of instances
- **1b.** IoU 0.5 is the loosest of the ten COCO thresholds; a detector that localises badly still scores well there, and the 0.50-to-0.95 average would be much lower
- **2.** Any one of: the number of images, the per-class AP distribution, the instance count per class, or the same metric averaged over the ten IoU thresholds

“The sample size” alone answers part 2. It is the single most informative thing missing from most reported scores.

What we did

1. Established that a label is not an answer when the question is *where* and *how many*
2. Derived IoU, and proved it is exactly zero for every pair of disjoint boxes — so as a loss it gives no gradient however far apart they are
3. Saw GIoU and CIoU repair that by adding a term that keeps moving
4. Built average precision from matching, and saw why its definition contains a maximum: precision is not monotone in the threshold, proved in Lecture 3, and the maximum is the repair
5. Read mAP as a mean over classes of a mean over IoU thresholds, and saw how much each averaging hides
6. Ran a detector, chose a score cut-off and defended it, applied non-maximum suppression, and looked at what a box cannot express

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 14

five, in the style of the written paper

Exercises — Lecture 14

1 Two boxes are disjoint and moved further apart. State what IoU and its derivative do, and why no learning rate repairs it. [5]

2 An IoU implementation without a clamp is tested on overlapping boxes only and passes. Give the input that breaks it, and the property test that would have caught it. [5]

3 Average precision is defined with a maximum inside it. State what that maximum repairs, and which earlier lecture proved the problem. [4]

Solutions on Lecture 15’s deck.

Exercises — Lecture 14 (continued)

4 mAP is a mean over classes of a mean over IoU thresholds. Give one thing each averaging hides. [4]

5 A team computes AP per image and averages those. State the direction of the error and its cause. [4]

Solutions on Lecture 15’s deck.

THE FIVE SET AT THE END OF LECTURE 13

Solutions Lecture 13

not part of this lecture

Solutions — Lecture 13

1. State the order in which a pretrained network's head must be replaced and its body frozen, and what…

✓ Freeze first, then replace. In the other order the new head is frozen too.

- A newly constructed module has `requires_grad=True`, so the freezing loop would turn it off
- The symptom is a loss that never moves, with nothing raising

2. Fine-tuning uses 10^{-4} on the pretrained block and 10^{-3} on the new head. Justify the difference in…

✓ The block already encodes something and the head starts from noise; one rate cannot suit both.

- A large step destroys what the block knows
- A small step leaves the random head untrained

Solutions — Lecture 13 (2)

3. Freezing a batch-norm layer's weights does not freeze the layer. State what still changes, and the call…

✓ The running statistics are buffers, updated in the forward pass; `.eval()` stops them.

- `requires_grad=False` governs gradients, not buffers
- A frozen feature extractor left in train mode still drifts

4. An augmented validation loader is used to select the epoch to keep. State the two things this costs, and the…

✓ The score becomes a random variable and the chosen epoch changes. Evaluate twice and require identical answers.

- Augmentation makes the metric depend on the evaluator's seed
- A deterministic function of fixed weights and fixed data does not wobble

Solutions — Lecture 13 (3)

5. A frozen-backbone probe is both more accurate and faster than training from scratch on 1,020 images. Explain…

✓ The backbone is a fitted function reused for free; the feature extraction pass must still be counted in the total.

- Only the head is fitted, so the variance problem moves to a dataset large enough to absorb it
- The probe is not free just because the head is

LECTURE 15 · GÉRON CH. 13

Time series

Stationarity, differencing and autocorrelation, derived
Why the naive forecast is hard to beat, and why a random split is invalid on autocorrelated data
Backtesting on a rolling origin

Run the Lecture 14 notebook first